For further details on the problem solutions, look inside each problem file for source code or output files.

p1
I did what the instructions told me to do and calculated the results with a Python script.
run chmod +x get_freq
run ./get_freq 100 > freq_output.txt
run python3 task1.py
or
run python3 task1.py > results.txt

AI was used to discover and use the statistics package for the mean and standard deviation calculations.

p2
I created a make file that does what is wanted, and automated the demonstration with a bash script.

‘# run chmod +x run_p2.sh
run ./run_p2.sh # this prints the results to the console

p3
I made a circular dependency like the problem described, and make made all the files still. It recognizes these circular dependencies.
You can just run “make” to get the results

~/coding/thpc/rechardt_aarni_ex1/p3$ make
make: Circular target3 <- target1 dependency dropped.
touch target3
touch target2
touch target1

p4

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 34.13      1.29     1.29                             __cos_fma
 30.42      2.44     1.15                             __sin_fma
 17.99      3.12     0.68                             __ieee754_exp_fma
 10.32      3.51     0.39        1   390.00   390.00  main
  5.82      3.73     0.22                             expf64
  1.32      3.78     0.05                             _init
```

The code has 2 inner loops. In the second one, we called math functions, and in the first one we didn't. So we can't know the exact time of the inner loops, but we can get pretty close and see where the majority of time was spent. As we see in the results, most of the time was spent outside of the main function, and in the math functions, around 88% of the time. So this means that only a small amount of time was spent in the first loop, and we have to remember that the 2nd loop also did the same work that the first loop did, with the additions and multiplications in main. The first loop may have taken around 5% of the execution time, which is equal to around 0.2 seconds. And the second inner loop took around 95% of the execution time.

commands for execution:
$ gcc -O0 -pg -g -static simpleprofiling.c -o simpleprog -lm
$ ./simpleprog
$ gprof ./simpleprog gmon.out > profiling_results.txt